

損保管理システム 基本アーキテクチャ図

～ 契約管理・事故対応の業務支援アプリ ～

設計書関係資料に基づく構成案

1. 全体アーキテクチャ概要

MVCパターン（Servlet/JSP）による責務分離

Browser

ユーザー操作
リクエスト送信



Controller

Servlet
リクエスト制御



Service

Logic / BO
業務ルール実行



DAO / DB

Data Access
MySQL連携

Controller (Servlet)

リクエスト受付、パラメータ取得、Serviceの呼び出し、画面遷移の指示

Service (Logic)

業務ルールのチェック、トランザクション管理、DAOの呼び出し

DAO (Data Access)

SQLの実行、検索結果をModel（Entity）へ変換

View (JSP)

JSTLを使用した画面表示、バッジ等の条件判定

2. 契約一覧・集計フロー

早期更改率の計算と一覧表示のプロセス

01

リクエスト受付 (PolicyListServlet)

GET /policies にて q（検索語）と tab（タブ）を受け取り、Serviceを呼び出し。

02

業務ロジック実行 (StatsService / PolicyService)

getAllPolicies()/searchPolicies() で一覧を取得し、getFiscalYearStats()/getMonthlyStats() で早期更改率を計算。

03

データアクセス (PolicyDao)

findAll() と search() を実行し、MySQLから情報を取得。

04

画面生成 (list.jsp)

JSTLで一覧をループ表示。満期20日前なら「要注意」バッジを付与。

集計ロジック

分母：当年度/当月の更新済み件数
分子：早期更改期間内の更新件数

アラート判定

満期20日前 ～ 満期日当日：要注意バッジ表示

3. 契約更新フロー

期間判定と満期日更新のプロセス

1

更新リクエスト (PolicyActionServlet)

POST /policies/renew にて対象IDを受け取り、PolicyServiceを呼び出し。

2

業務ルール検証 (PolicyService)

ステータスが「契約中」かつ「更新可能期間内」であることをチェック。

3

データ更新 (PolicyDao)

旧満期日を退避し、満期日を1年延長。更新日時を記録してDBを更新。

主要な業務ロジック

- ▶ **更新可能期間:** 満期日の2ヶ月前 ~ 満期日当日
- ▶ **満期日計算:** 開始日のちょうど1年後（うるう年考慮）
- ▶ **更新取消:** 更新操作を行った当日のみ実行可能
- ▶ **例外処理:** ルール違反時は BusinessException をスロー

※更新後は詳細画面へリダイレクトし、最新の契約状態を表示します。

4. 事故ステータス変更フロー

ステータス遷移と対応日時の記録プロセス



Controller / Service

AccidentActionServlet がリクエストを受け、AccidentService で遷移ルールをチェック。

対応日時の更新

ステータス変更時、同時に last_contacted_at（最終対応日時）を現在時刻に更新。

⚠️ 滞留判定ルール

以下のいずれかに該当する場合、一覧画面に「滞留」バッジを表示：

- ・ 最終対応日時が未記録（未対応）
- ・ 最終対応日時から 7日以上経過している（※完了済みを除く）

5. データベース構造とリレーション

MySQL 8.x における主要テーブル構成

